

Подсказки и типовые ошибки

Здесь собраны типовые ошибки, с которыми часто сталкиваются разработчики при анализе производительности с помощью flamegraph и criterion, а также рекомендации по их устранению.

- 1 Ошибка 1. Flamegraph показывает только адреса, а не имена функций**
В flamegraph видны только шестнадцатеричные адреса вместо имён функций. Это происходит, когда бинарь собран без debug-символов или они были удалены.
Добавьте в Cargo.toml секцию `[profile.release]` с `debug = 1`, пересоберите проект через `cargo build --release` и убедитесь, что используете release-профиль для профилирования.
- 2 Ошибка 2. Стеки вызовов неполные или рванные**
В flamegraph видны усечённые стеки, некоторые функции пропускаются. Это происходит из-за агрессивных оптимизаций компилятора, которые удаляют frame pointers или инлайнят функции. Включите сохранение frame pointers через `RUSTFLAGS="-C force-frame-pointers=yes" cargo build --release` и используйте более новые версии perf и flamegraph, которые лучше работают с DWARF.
- 3 Ошибка 3. Фокус на узких блоках вместо широких**
Вы пытаетесь оптимизировать функции, которые занимают 1–2% времени. Сначала оптимизируйте функции, которые занимают 20%+ времени. Широкие блоки в flamegraph — это горячие точки, узкие блоки можно игнорировать до тех пор, пока не оптимизированы широкие.
- 4 Ошибка 4. При работе с criterion компилятор оптимизирует весь код, результаты нереалистичны**
Бенчмарк показывает 0 ns/iter или нереально маленькие значения, потому что компилятор выкидывает код, результат которого не используется. Всегда используйте black_box для входных данных: `b.iter(|| function(black_box(input)))`. Используйте black_box для результатов, если они не возвращаются: `b.iter(|| black_box(function(input)))`.
- 5 Ошибка 5. При работе с criterion бенчмарк показывает "Performance has regressed", но код стал быстрее**
Criterion говорит об ухудшении, хотя вы оптимизировали код. Возможные причины: вы изменили входные данные между запусками, система была загружена во время одного из запусков, или код действительно стал медленнее из-за ошибки в оптимизации. Убедитесь, что входные данные одинаковые, запустите бенчмарк несколько раз и посмотрите на средние значения, проверьте код оптимизации на ошибки.
- 6 Ошибка 6. При работе с criterion бенчмарки не компилируются или не запускаются**
cargo bench выдаёт ошибки компиляции или не находит бенчмарки. Убедитесь, что в Cargo.toml есть секция `[[bench]]` с `name = "your_bench"` и `harness = false`. Проверьте, что файл бенчмарка находится в benches/your_bench.rs, функции, которые вы бенчмарките, публичные (pub fn) и импорты в файле бенчмарка корректны.